NFL game predictor

Keegan Prunty

2025-11-12

Objective

The goal of this project is to predict whether a home team will win an NFL regular season game using cumulative information on the team's season. This was done so that it can predict games that have not occured yet based on pre-kickoff information. We will create a data set based on NFL season to date team stats and fit a Logistic Regression, along with feature selection for the best model.

Data Background

nflreadr::load_schedules(2024) for game-level results (teams, scores, week, game_id).

nflreadr::load_pbp(2024) for play-by-play data to compute drive-based stats, third-down rates, turnovers, and EPA.

(For this specific model we will be using 2024 regular season data)

Data Manipulation and Cleaning

We build a table with one row per team per game (team_game) containing: 1. Points for / against 2. Win indicator 3. Turnover margin (takeaways - giveaways) 4. Drive scoring rate (scoring drives / total drives) 5. Time of possession (seconds) 6. Third-down success rate 7. Offensive EPA per play

These chosen stats are often key indicators of team success to an avid NFL fan.

```
# packages
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(nflreadr)
library(slider)
```

```
## Warning: package 'slider' was built under R version 4.5.2
```

```r
library(tidyr)

season <- 2024

# ---------- 1) Load schedules & pbp ----------
sched <- load_schedules(season) %>%
  filter(game_type == "REG") %>%
  select(game_id, season, week, home_team, away_team, home_score, away_score)

pbp <- load_pbp(season) %>%
  # keep regular season only
  inner_join(sched %>% select(game_id), by = "game_id")

# ---------- 2) Build team-game base (points for/against) ----------
team_game <- bind_rows(
  sched %>% transmute(season, week, game_id,
                      team = home_team, opp_team = away_team,
                      pts_for = home_score, pts_against = away_score,
                      home = 1L),
  sched %>% transmute(season, week, game_id,
                      team = away_team, opp_team = home_team,
                      pts_for = away_score, pts_against = home_score, home = 0L)
) %>%
  arrange(season, team, week) %>%
  group_by(season, team) %>%
  mutate(
    win = as.integer(pts_for > pts_against),
    cum_wins = lag(cumsum(win)),
    games_played = lag(row_number() - 1L),
    win_pct = ifelse(games_played > 0, cum_wins / games_played, NA_real_),
    last3_win_pct = lag(slide_dbl(win, mean, .before = 3, .complete = FALSE))
  ) %>%
  ungroup()

# ---------- 3) Turnovers: giveaways (offense) & takeaways (defense) ----------
turnovers_off <- pbp %>%
  filter(!is.na(posteam)) %>%
  group_by(game_id, team = posteam) %>%
  summarise(
    giveaways = sum(replace_na(interception, 0)) +
      sum(replace_na(fumble_lost, 0)),
    .groups = "drop"
  )

# defensive takeaways = opponent's giveaways (map via opponent)
turnovers_def <- turnovers_off %>%
  rename(opp_team = team, takeaways = giveaways)
```

```r
team_game <- team_game %>%
  left_join(turnovers_off, by = c("game_id","team")) %>%
  left_join(turnovers_def, by = c("game_id","opp_team")) %>%
  mutate(
    giveaways = replace_na(giveaways, 0),
    takeaways = replace_na(takeaways, 0),
    to_margin = takeaways - giveaways
  ) %>%
  group_by(season, team) %>%
  mutate(
    cum_to_margin = lag(cumsum(to_margin)),       # cumulative turnover margin
    to_margin_per_game = ifelse(games_played > 0, cum_to_margin / games_played, NA_real_)
  ) %>% ungroup()

# ---------- 4) Scoring % (drive-based) ----------
# Build drive-level table (offensive drives only) from pbp
# helper: parse "mm:ss" to seconds (NA-safe)
parse_mmss <- function(x) {
  x <- ifelse(is.na(x) | x == "", NA, x)
  parts <- strsplit(x, ":", fixed = TRUE)
  as.numeric(vapply(parts, function(p)
    if (length(p) == 2) as.integer(p[1]) * 60 +
      as.integer(p[2]) else NA_real_, 1.0))
}

drives <- pbp %>%
  filter(!is.na(posteam), !is.na(drive)) %>% #keep offensive plays with id
  group_by(game_id, team = posteam, drive) %>%
  summarise(
    # scoring: any play on this drive marked as ended-with-score
    drive_scored = as.integer(any(coalesce(drive_ended_with_score, 0) == 1)),
    # take the last non-NA TOP string for the drive (it's "mm:ss")
    drive_top_mmss = dplyr::last(na.omit(drive_time_of_possession)),
    .groups = "drop_last"
  ) %>%
  # parse mm:ss to numeric seconds
  mutate(drive_seconds = parse_mmss(drive_top_mmss)) %>%
  # collapse to team-game
  group_by(game_id, team) %>%
  summarise(
    drives_total  = n(),
    drives_scored = sum(drive_scored, na.rm = TRUE),
    top_seconds   = sum(coalesce(drive_seconds, 0), na.rm = TRUE),
    .groups = "drop"
  )

team_game <- team_game %>%
  left_join(drives, by = c("game_id","team")) %>%
  mutate(
    drives_total = replace_na(drives_total, 0),
    drives_scored = replace_na(drives_scored, 0),
    top_seconds = replace_na(top_seconds, 0),
    score_pct_game = ifelse(drives_total > 0, drives_scored / drives_total,
```

```
                              NA_real_)
  ) %>%
  group_by(season, team) %>%
  mutate(
    score_pct_accum = lag(cummean(score_pct_game)),          # cum scoring %
    top_per_game = lag(cumsum(top_seconds) / (games_played)) # cumavg time
  ) %>% ungroup()

# ---------- 5) 3rd-down success rate (offense) ----------

# --- Offensive 3rd-down per team-game ---
third_off <- pbp %>%
  filter(!is.na(posteam), down == 3) %>%
  group_by(game_id, team = posteam) %>%
  summarise(
    third_conv = sum(as.integer(coalesce(third_down_converted, 0))),
    third_fail = sum(as.integer(coalesce(third_down_failed, 0))),
    third_att  = third_conv + third_fail,
    third_sr_game = ifelse(third_att > 0, third_conv / third_att, NA_real_),
    .groups = "drop"
  )

# --- Defensive 3rd-down per team-game (allowed) ---
third_def <- pbp %>%
  filter(!is.na(defteam), down == 3) %>%
  group_by(game_id, team = defteam) %>%
  summarise(
    third_conv_allowed = sum(as.integer(coalesce(third_down_converted, 0))),
    third_fail_forced  = sum(as.integer(coalesce(third_down_failed, 0))),
    third_att_def      = third_conv_allowed + third_fail_forced,
    third_sr_allowed_game = ifelse(third_att_def > 0, third_conv_allowed /
                                     third_att_def, NA_real_),
    .groups = "drop"
  )

# Join into your team_game table and make cumulative (season-to-date) versions
team_game <- team_game %>%
  left_join(third_off, by = c("game_id","team")) %>%
  left_join(third_def, by = c("game_id","team")) %>%
  group_by(season, team) %>%
  mutate(
    third_sr_accum_off = lag(cummean(third_sr_game)),          # offense S2D
    third_sr_accum_def = lag(cummean(third_sr_allowed_game)),  # defense S2D
    third_att_cum_off  = lag(cumsum(coalesce(third_att, 0))),  #count
    third_att_cum_def  = lag(cumsum(coalesce(third_att_def, 0)))
  ) %>%
  ungroup()

# ---------- 6) EPA (offense & defense), per game and cumulative ----------
epa_game <- pbp %>%
  filter(!is.na(posteam)) %>%
  group_by(game_id, team = posteam) %>%
  summarise(epa_off_play = mean(epa, na.rm = TRUE), .groups = "drop")
```

```r
epa_allowed_game <- pbp %>%
  filter(!is.na(defteam)) %>%
  group_by(game_id, team = defteam) %>%
  summarise(epa_def_allowed = mean(epa, na.rm = TRUE), .groups = "drop")

team_game <- team_game %>%
  left_join(epa_game, by = c("game_id","team")) %>%
  left_join(epa_allowed_game, by = c("game_id","team")) %>%
  group_by(season, team) %>%
  mutate(
    epa_off_cum = lag(cummean(epa_off_play)),        # cum offensive EPA/play
    epa_def_cum = lag(cummean(epa_def_allowed))      # cum defensive EPA/play
  ) %>% ungroup()

# ---------- 7) Opponent pregame win% and cumulative SOS ----------
opp_wp <- team_game %>%
  select(season, week, team, win_pct) %>%
  rename(opp_team = team, opp_win_pct_before = win_pct)

team_game <- team_game %>%
  left_join(opp_wp, by = c("season","week","opp_team")) %>%
  group_by(season, team) %>%
  mutate(
    sos_accum = lag(cummean(opp_win_pct_before))     # difficulty of schedule
  ) %>% ungroup()

# ---------- add cumulative (pregame) metrics before feature export ----------
team_game <- team_game %>%
  mutate(week = as.integer(week)) %>%
  arrange(season, team, week) %>%
  group_by(season, team) %>%
  mutate(
    # games / wins before current game
    gp_before   = lag(row_number(), default = 0L),
    wins_cum    = cumsum(win),
    wins_before = lag(wins_cum, default = 0L),
    win_pct     = ifelse(gp_before > 0, wins_before / gp_before, NA_real_),

    # cumulative points for/against (pregame PPG)
    pts_for_cum_before     = lag(cumsum(pts_for), default = 0),
    pts_against_cum_before = lag(cumsum(pts_against), default = 0),
    ppg_for_cum            = ifelse(gp_before > 0, pts_for_cum_before /
                                    gp_before, NA_real_),
    ppg_against_cum        = ifelse(gp_before > 0, pts_against_cum_before /
                                    gp_before, NA_real_),

    # cumulative turnover margin
    to_margin_cum_before   = lag(cumsum(to_margin), default = 0),
    to_margin_per_game     = ifelse(gp_before > 0, to_margin_cum_before /
                                    gp_before, NA_real_),

    # cumulative scoring %, TOP, third downs, EPA, etc.
    drives_scored_before   = lag(cumsum(coalesce(drives_scored, 0)),
```

```
                                  default = 0),
    drives_total_before    = lag(cumsum(coalesce(drives_total, 0)),
                                  default = 0),
    score_pct_accum        = ifelse(drives_total_before > 0,
                                    drives_scored_before / drives_total_before,
                                    NA_real_),

    top_seconds_before     = lag(cumsum(coalesce(top_seconds, 0)), default = 0),
    top_per_game           = ifelse(gp_before > 0, top_seconds_before /
                                      gp_before, NA_real_),

    third_conv_cum_off     = lag(cumsum(coalesce(third_conv, 0)), default = 0),
    third_att_cum_off      = lag(cumsum(coalesce(third_att, 0)),  default = 0),
    third_sr_accum_off     = ifelse(third_att_cum_off > 0, third_conv_cum_off /
                                      third_att_cum_off, NA_real_),

    third_conv_cum_def     = lag(cumsum(coalesce(third_conv_allowed, 0)),
                                  default = 0),
    third_att_cum_def      = lag(cumsum(coalesce(third_att_def, 0)),
                                  default = 0),
    third_sr_accum_def     = ifelse(third_att_cum_def > 0, third_conv_cum_def /
                                      third_att_cum_def, NA_real_),

    epa_off_sum_before     = lag(cumsum(coalesce(epa_off_play, NA_real_)),
                                  default = NA),
    epa_off_cnt_before     = lag(cumsum(!is.na(epa_off_play)), default = 0),
    epa_off_cum            = ifelse(epa_off_cnt_before > 0, epa_off_sum_before /
                                      epa_off_cnt_before, NA_real_),

    epa_def_sum_before     = lag(cumsum(coalesce(epa_def_allowed, NA_real_)),
                                  default = NA),
    epa_def_cnt_before     = lag(cumsum(!is.na(epa_def_allowed)), default = 0),
    epa_def_cum            = ifelse(epa_def_cnt_before > 0, epa_def_sum_before /
                                      epa_def_cnt_before, NA_real_)
  ) %>%
  ungroup()

# ---------- fix ppg ----------
team_game <- team_game %>%
  mutate(week = as.integer(week)) %>%
  arrange(season, team, week) %>%
  group_by(season, team) %>%
  mutate(
    gp_before        = lag(row_number(), default = 0L), #games before
    wins_cum         = cumsum(win),
    wins_before      = lag(wins_cum, default = 0L),
    win_pct          = ifelse(gp_before > 0, wins_before / gp_before, NA_real_)
  ) %>% ungroup()

team_game <- team_game %>%
  arrange(season, team, week) %>%
  group_by(season, team) %>%
  mutate(
```

```
    # games played before this one
    gp_before = lag(row_number(), default = 0L),

    # cumulative totals before this week
    pts_for_before     = lag(cumsum(pts_for), default = 0),
    pts_against_before = lag(cumsum(pts_against), default = 0),

    # pregame average (PPG)
    ppg_for_cum     = ifelse(gp_before > 0, pts_for_before / gp_before,
                             NA_real_),
    ppg_against_cum = ifelse(gp_before > 0, pts_against_before / gp_before,
                             NA_real_)
  ) %>%
  ungroup()



# ---------- 9) Final: select what you need ----------
team_game_features <- team_game %>%
  transmute(
    season, week, game_id, team, opp_team, home,
    pts_for, pts_against, ppg_for_cum, ppg_against_cum, win, win_pct,
    last3_win_pct,
    giveaways, takeaways, to_margin, cum_to_margin, to_margin_per_game,
    drives_total, drives_scored,
    score_pct_game = coalesce(score_pct_game, NA_real_),
    score_pct_accum = coalesce(score_pct_accum, NA_real_),
    top_seconds = coalesce(top_seconds, NA_real_),
    top_per_game = coalesce(top_per_game, NA_real_),

    third_sr_game_off  = coalesce(third_sr_game, NA_real_),
    third_sr_accum_off = coalesce(third_sr_accum_off, NA_real_),
    third_sr_game_def  = coalesce(third_sr_allowed_game, NA_real_),
    third_sr_accum_def = coalesce(third_sr_accum_def, NA_real_),

    epa_off_play  = coalesce(epa_off_play, NA_real_),
    epa_off_cum   = coalesce(epa_off_cum, NA_real_),
    epa_def_allowed = coalesce(epa_def_allowed, NA_real_),
    epa_def_cum   = coalesce(epa_def_cum, NA_real_),

    opp_win_pct_before = coalesce(opp_win_pct_before, NA_real_),
    sos_accum = coalesce(sos_accum, NA_real_)
  )
```

This is our model approach using these final mathcup diferentials.

```
# Create home/away feature tables for joining onto schedule
home_feats <- team_game_features %>%
  transmute(
    game_id,
    home_team = team,
    home_ppg_for_cum       = ppg_for_cum,
    home_ppg_against_cum   = ppg_against_cum,
```

```
    home_epa_off_cum       = epa_off_cum,
    home_to_margin_per_game= to_margin_per_game,
    home_score_pct_accum   = score_pct_accum,
    home_third_sr_accum_off= third_sr_accum_off,
    home_top_per_game      = top_per_game,
    home_win_pct           = win_pct,
    home_last3_win_pct     = last3_win_pct
  )

away_feats <- team_game_features %>%
  transmute(
    game_id,
    away_team = team,
    away_ppg_for_cum       = ppg_for_cum,
    away_ppg_against_cum   = ppg_against_cum,
    away_epa_off_cum       = epa_off_cum,
    away_to_margin_per_game= to_margin_per_game,
    away_score_pct_accum   = score_pct_accum,
    away_third_sr_accum_off= third_sr_accum_off,
    away_top_per_game      = top_per_game,
    away_win_pct           = win_pct,
    away_last3_win_pct     = last3_win_pct
  )

matchups <- sched %>%
  left_join(home_feats, by = c("game_id","home_team")) %>%
  left_join(away_feats, by = c("game_id","away_team")) %>%
  mutate(
    # Target variable: 1 if home team won
    home_win = as.integer(home_score > away_score),

    # Feature differentials (home - away)
    d_ppg_for_cum     = home_ppg_for_cum     - away_ppg_for_cum,
    d_ppg_against_cum = home_ppg_against_cum - away_ppg_against_cum,
    d_epa_off_cum     = home_epa_off_cum     - away_epa_off_cum,
    d_to_margin_pg    = home_to_margin_per_game - away_to_margin_per_game,
    d_score_pct       = home_score_pct_accum - away_score_pct_accum,
    d_third_sr_off    = home_third_sr_accum_off - away_third_sr_accum_off,
    d_top_per_game    = home_top_per_game    - away_top_per_game,
    d_win_pct         = home_win_pct         - away_win_pct,
    d_last3_win_pct   = home_last3_win_pct   - away_last3_win_pct
  )
```

```
#picking the features I want and dropping NA for early games with missing stats
model_data <- matchups %>%
  filter(!is.na(d_ppg_for_cum)) %>%
  select(
    home_win,
    d_ppg_for_cum,
    d_ppg_against_cum,
    d_epa_off_cum,
    d_to_margin_pg,
    d_score_pct,
```

```
    d_third_sr_off,
    d_top_per_game,
    d_win_pct,
    d_last3_win_pct
  )
```

Logistic Regression

Let's create our model and take a look at our features.

```
#splitting data
set.seed(42)
train_index <- sample(seq_len(nrow(model_data)), size = 0.8 * nrow(model_data))
train_data <- model_data[train_index, ]
test_data  <- model_data[-train_index, ]


#fitting model logistoc regression
model_logit <- glm(home_win ~ ., data = train_data, family = binomial)
summary(model_logit)
```

```
##
## Call:
## glm(formula = home_win ~ ., family = binomial, data = train_data)
##
## Coefficients:
##                    Estimate Std. Error z value Pr(>|z|)
## (Intercept)        0.0162125  0.1553351   0.104   0.9169
## d_ppg_for_cum      0.0137165  0.0505017   0.272   0.7859
## d_ppg_against_cum -0.0156412  0.0344666  -0.454   0.6500
## d_epa_off_cum      5.6495993  3.0426930   1.857   0.0633 .
## d_to_margin_pg     0.0316958  0.1785856   0.177   0.8591
## d_score_pct       -1.5749627  2.9473768  -0.534   0.5931
## d_third_sr_off    -1.7442820  2.3360018  -0.747   0.4552
## d_top_per_game    -0.0005766  0.0009091  -0.634   0.5259
## d_win_pct          1.3933083  1.1271456   1.236   0.2164
## d_last3_win_pct    0.1450096  0.7161592   0.202   0.8395
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 282.80  on 203  degrees of freedom
## Residual deviance: 245.03  on 194  degrees of freedom
## AIC: 265.03
##
## Number of Fisher Scoring iterations: 4
```

```
exp(coef(model_logit))
```

```
##       (Intercept)     d_ppg_for_cum d_ppg_against_cum     d_epa_off_cum
```

```
##         1.0163446         1.0138110         0.9844805       284.1775616
##    d_to_margin_pg       d_score_pct    d_third_sr_off    d_top_per_game
##         1.0322034         0.2070153         0.1747704         0.9994236
##         d_win_pct   d_last3_win_pct
##         4.0281544         1.1560506
```

```
test_data$pred_prob <- predict(model_logit, newdata = test_data,
                               type = "response")
test_data$pred_win  <- ifelse(test_data$pred_prob > 0.5, 1, 0)
```

As you can see non of the features seem to be significant above and we can run into issues using many predictors like this, so lets use methods to select the beast features for our model.

LASSO Logistic Regression for Feature Selection

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'Matrix'
```

```
## The following objects are masked from 'package:tidyr':
##
##     expand, pack, unpack
```

```
## Loaded glmnet 4.1-10
```

```
## [1] 0.02604824
```

```
## 10 x 1 sparse Matrix of class "dgCMatrix"
##                    lambda.1se
## (Intercept)       0.08971313
## d_ppg_for_cum     .
## d_ppg_against_cum .
## d_epa_off_cum     1.82461058
## d_to_margin_pg    .
## d_score_pct       .
## d_third_sr_off    .
## d_top_per_game    .
## d_win_pct         0.44266375
## d_last3_win_pct   .
```

```
##
## Call:
## glm(formula = home_win ~ d_epa_off_cum + d_win_pct, family = binomial,
##     data = train_data)
##
## Coefficients:
##               Estimate Std. Error z value Pr(>|z|)
## (Intercept)    0.02883    0.15333   0.188  0.85088
## d_epa_off_cum  3.64876    1.51881   2.402  0.01629 *
```

```
## d_win_pct      1.65429    0.60147   2.750  0.00595 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 282.80  on 203  degrees of freedom
## Residual deviance: 247.45  on 201  degrees of freedom
## AIC: 253.45
##
## Number of Fisher Scoring iterations: 3
```

```
##   (Intercept) d_epa_off_cum     d_win_pct
##      1.029245     38.427145      5.229345
```

```
## [1] 0.7115385
```

```
library(pROC)
```

```
## Type 'citation("pROC")' for a citation.
```

```
##
## Attaching package: 'pROC'
```

```
## The following objects are masked from 'package:stats':
##
##     cov, smooth, var
```

```
# Reduced model ROC
roc_simple <- roc(test_data$home_win, test_data$pred_prob_simple)
```

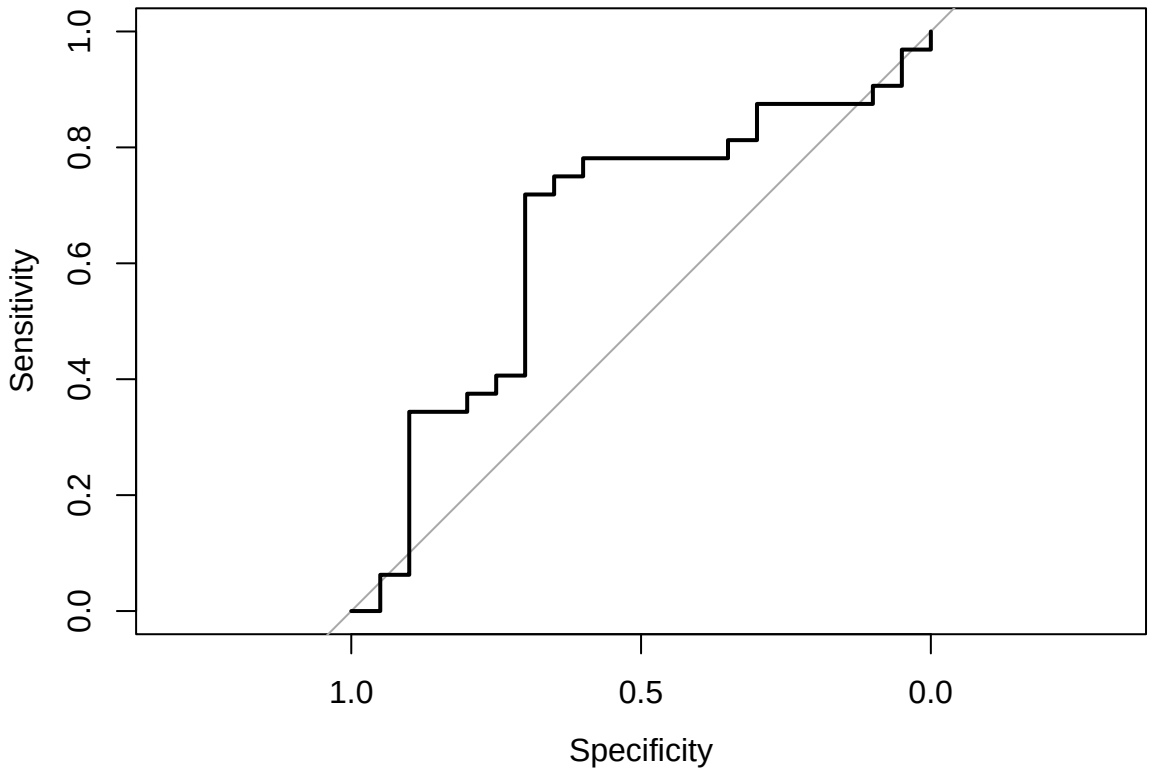
```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
auc(roc_simple)
```

```
## Area under the curve: 0.6547
```

```
plot(roc_simple)
```




```
coords(roc_simple, "best", best.method = "youden")
```

```
##   threshold specificity sensitivity
## 1 0.5153841         0.7     0.71875
```

Results

Even though many individual predictors were not statistically significant, the model still predicts well because the predictors collectively capture team strength and matchup edges. Based on our LASSO method choosing difference in EPA and difference in win percentage we determine the others to be redundant because the simpler model will return similar accuracy with a simpler model.

Limitations

Using only 2024 season limits the sample size and can make coefficients unstable. Also some improvements could be made on the model that can also be key to a team winning or not, such as key injuries, weather, mathcup histtory, and team strength, such as if team A has a very poor run defense but better record, and team B has a very strong run offense but a worse record, this could predict team A would have a good chance of winning but this would be a very key stat to the game that my model is missing. Things likeweather and past injury reports as well as team strength are very hard to obtainfor past games but in a larger project including things like these could add keyfeatures to a win predicting model.